

Lab 06 — Build the Course Assistant and Drive It with /goal

Agentic AI Applications with Codex · Day 1 · Codex

Course: Agentic AI Applications with Codex (TGS-2023041081)

Day 1 · Topic 2 · about 35 minutes · slides 70–74

Surface: Codex

Features: In-browser RAG · golden-question evaluation · /goal

The story so far

Grace wants the assistant live before term — but only if it is right. A wrong allergy answer is a liability. Build it, measure it against 30 questions real customers asked, and do not stop until it scores 30 out of 30.

Your goal

A chatbot you cannot measure is a chatbot you cannot trust. Build it, score it, then let /goal push the score to 30/30.

You'll build

A working course assistant that passes 30/30 golden questions

What is in this folder

- `assets/golden-questions.csv`
- `assets/assistant-spec.md`
- `solution/` — reference files from the verified build
- `prompts.md` / `prompts.pdf` — every prompt, ready to paste
- `evidence/checklist.md` — what to capture as proof

Step by step

1. **Vendor SQLite** — Copy `node_modules/@sqlite.org/sqlite-wasm/dist/index.mjs` and `sqlite3.wasm` into `vendor/sqlite-wasm/`.
2. **Build the assistant** — Copy `golden-questions.csv` to `eval/`, then paste Prompt A.
3. **Score the baseline** — Run `npm run eval`. Write down the score.
4. **Set a goal** — Paste Prompt B. Check progress with `/goal`; use `/goal pause` if it wanders.
5. **Review the diff** — Confirm `golden-questions.csv` is unchanged: `git diff eval/`.
6. **Try it in the browser** — Ask about nut allergies, the cheapest course, and the weather.

The prompts

PROMPT A — Codex

Build the course assistant, following assistant-spec.md.

- js/rag.js: buildQuery (quote every term, join with OR — never pass raw text to MATCH), search(db, text, k) with bm25 weights favouring title, and extractiveAnswer(hits).
- js/chat.js: a chat panel that loads data/academy.db into SQLite WASM with sqlite3_deserialize, answers from the top hit, and lists its sources as links.
- No results → a polite refusal with our contact.
- Render all text with textContent.
- Add npm run eval: ask every question in eval/golden-questions.csv and pass it when the top 3 results include the expected source and text. Print the score, e.g. 30/30.

PROMPT B — /goal

/goal npm run eval reports 30/30 passed.

Rules:

- Never edit eval/golden-questions.csv.
- Improve js/rag.js or the kb/ documents only.
- Questions about the cheapest, most expensive or under S\$X need the courses table, not search.
- Off-topic questions must return no results.
- After each change, run npm run eval and report the score.

Check your work

- ☐ The assistant runs in the browser with no server code.
- ☐ You recorded a baseline score before /goal.
- ☐ npm run eval ends at 30/30 passed.
- ☐ git diff shows eval/golden-questions.csv unchanged.
- ☐ Every answer lists its sources as links.
- ☐ "What is the weather tomorrow?" gets the refusal.
- ☐ "What is the cheapest course?" answers CUL-210, S\$160.

If it goes wrong

- **Words like "when" and "how long" miss** — They never appear in the documents. Map them to the words that do: "intakes", "duration". That is query expansion.

Stretch

- Add a question the bot fails, then fix the retrieval, not the question.

Why it matters: The golden set is your exam paper. Changing it to pass is cheating — and the hooks lab will catch it.

Next

Lab 7 — Add ChatGPT Mode, Then Try to Break It. Keep what you built — the next lab starts from it.

Safety

Use only this lab's synthetic data and your own accounts. Never paste a real API key, password or personal data into a prompt or a file. Every email address in the data must resolve to your own inbox. Nothing is published or sent without your explicit approval.